

Software Requirement Specification (SRS)

Flight Fare Prediction System Using Machine Learning

Core Python & PostgreSQL with ML

Guide: Swarup Kumar Saha

Webskitters Academy

1 July 2026

Submitted By

Dion James Smith

Batch-467

1. Introduction

Airline ticket prices vary frequently due to factors such as travel demand, seasonal trends, airline competition, holidays, and flight schedules. As a result, passengers often find it difficult to determine the best time to book a flight at a reasonable price.

The Flight Fare Prediction System is a Machine Learning-based application that estimates the expected airfare using historical flight data. The system analyzes various flight attributes, including airline, source, destination, journey date, departure time, arrival time, flight duration, and the number of stops.

The project follows a structured Machine Learning workflow consisting of data preprocessing, feature engineering, model training, evaluation, and prediction. It is developed using Object-Oriented Programming principles and MVC architecture to ensure modularity, maintainability, and scalability.

2. Problem Statement

Airfare is highly dynamic and can change several times before the departure date. Since passengers have limited knowledge of pricing trends, they often struggle to estimate whether the current ticket price is reasonable or whether waiting for a better price would be beneficial.

The absence of a reliable fare estimation system makes travel planning more difficult and may lead to unnecessary travel expenses. A predictive solution that analyzes historical flight information can assist users in making more informed booking decisions.

3. Business Problem

The airline industry operates in a highly competitive environment where ticket prices fluctuate continuously based on demand, route popularity, booking patterns, travel seasons, and operational factors. Customers frequently compare multiple airlines before making a booking, but they lack an estimate of what a fair ticket price should be.

This project addresses the problem by developing a Machine Learning model capable of predicting flight fares using historical data. The prediction assists travelers in estimating expected ticket prices, improving travel planning, and supporting better financial decision-making before booking a flight.

4. Objectives

The primary objectives of the proposed system are:

- Develop a Machine Learning model capable of predicting flight fares accurately.
- Perform data cleaning and preprocessing on the historical flight dataset.
- Apply feature engineering techniques to improve prediction quality.
- Train and evaluate multiple regression algorithms for performance comparison.
- Save the trained model for future predictions using Pickle.
- Design the application using MVC architecture and Object-Oriented Programming concepts.
- Implement authentication with separate roles for administrators and users.
- Generate reports and maintain prediction history for future reference.

5. Scope

The scope of the project is limited to predicting domestic flight fares using historical flight data provided in the dataset. The system allows administrators to train and manage the Machine Learning model, while authenticated users can estimate ticket prices by entering flight details through the application. **User information, prediction history, and application records are maintained using a PostgreSQL database.**

The project focuses on demonstrating the complete Machine Learning pipeline, including preprocessing, model training, prediction, and evaluation. Although the system does not retrieve live airline pricing, its modular design allows future integration with real-time flight APIs and web-based applications.

6. Functional Requirements

The system shall provide the following functional features:

Administrator

- Secure login and logout.
- Upload and validate the flight dataset.
- Perform data cleaning and feature engineering.
- Train and evaluate Machine Learning models.
- Compare multiple regression algorithms.

- Save the trained model (.pkl).
- Generate analytical reports.
- Create and manage user accounts.
- View prediction history.

User

- Secure login and logout.
- Enter flight details for fare prediction.
- View predicted airfare.
- View previous prediction history.
- Export prediction reports.

System

- Validate all user inputs.
- Store trained Machine Learning models.
- Store and retrieve users, prediction history, and model information using PostgreSQL.
- Generate reports in CSV and PDF formats.
- Display prediction results with appropriate error handling.

7. Non-Functional Requirements

The system shall satisfy the following non-functional requirements to ensure reliability, performance, and maintainability throughout its lifecycle.

Performance

- Generate fare predictions within a few seconds after receiving valid input.
- Handle large historical datasets efficiently during model training.
- Optimize memory usage while processing data.

Reliability

- Produce consistent predictions for the same input when using the same trained model.
- Prevent application failure through proper exception handling and validation.
- Preserve prediction history and trained models without data loss.

Security

- Allow access only to authenticated users.
- Restrict administrative operations to authorized administrators.
- Validate all user inputs to prevent invalid or malicious data.

Maintainability

- Follow MVC architecture and separation of concerns.
- Develop modules that are loosely coupled and highly cohesive.
- Write reusable and modular code using Object-Oriented Programming principles.

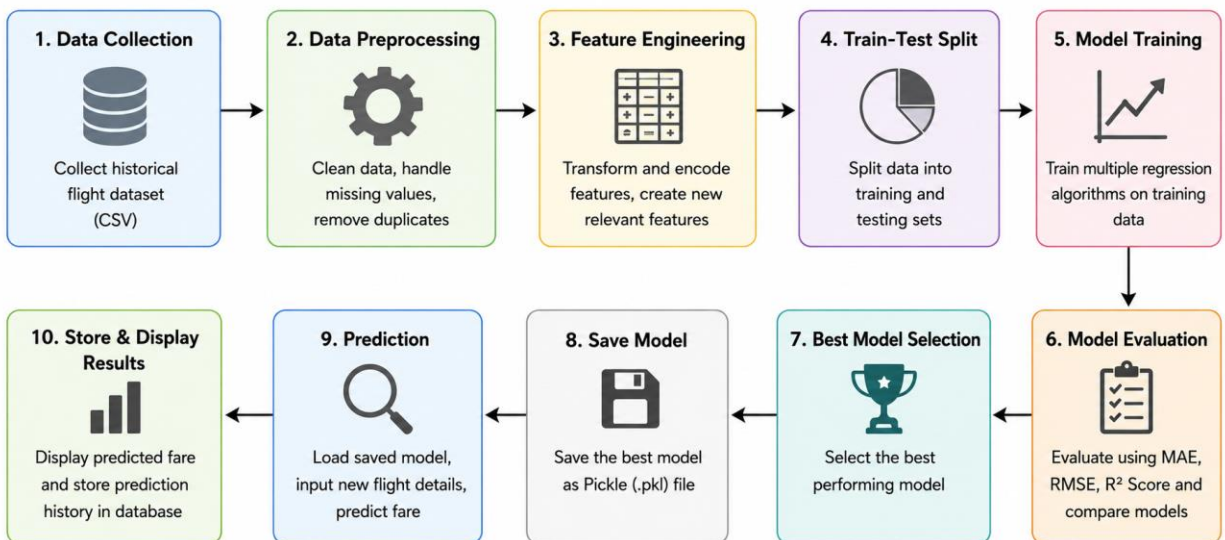
Scalability

- Support future integration with live flight APIs.
- Allow additional Machine Learning algorithms to be incorporated with minimal code changes.

8. System Architecture

- The system follows the **Model-View-Controller (MVC)** architectural pattern.
- View handles user interaction through a menu-driven interface.
- Controller receives user requests and coordinates between View and Services.
- Services contain the business logic for data processing, model training, prediction, and reporting.
- Model manages data access and persistence in the PostgreSQL database.
- Administrator module manages dataset upload, model training, evaluation, and report generation.
- User module allows users to input flight details and get fare predictions.
- Trained model is saved as a **Pickle (.pkl)** file and loaded during prediction.
- Modules communicate through well-defined interfaces, ensuring loose coupling and high cohesion.

9. Machine Learning Workflow



10. Technologies Used

The project is developed using the following technologies:

Technology	Purpose
Python	Core programming language
Pandas	Data manipulation and preprocessing
NumPy	Numerical computations
Matplotlib	Data visualization and graphical analysis
Scikit-Learn	Machine Learning model development
Pickle	Saving and loading trained models
PostgreSQL	Database for storing users, predictions, model info & reports
CSV	Storage of historical flight data
Object-Oriented Programming	Modular and reusable software design
MVC Architecture	Separation of concerns within the application

11. Dataset Description

The project utilizes a historical flight dataset containing details of previous airline bookings and their corresponding ticket fares. Each record represents a completed flight and serves as training data for the Machine Learning model.

The dataset contains attributes such as airline name, source city, destination city, journey date, departure time, arrival time, flight duration, total number of stops, additional information, and the corresponding flight fare. Flight fare is treated as the target variable, while the remaining attributes are used as input features for prediction.

Before model training, the dataset undergoes preprocessing to handle missing values, remove duplicate entries, encode categorical variables, and transform time-based features into numerical values suitable for Machine Learning algorithms.

12. OOP Concepts Used

The project applies the following Object-Oriented Programming concepts:

- **Class and Object:** Represent system entities such as User, Admin, Flight, and Prediction.
- **Encapsulation:** Protect class data through controlled access using methods.
- **Inheritance:** Allow the Admin class to inherit common properties from the User class.
- **Abstraction:** Define common behavior using abstract base classes.
- **Polymorphism:** Enable different implementations of common methods where required.
- **Method Overriding:** Customize inherited methods for specific functionality.
- **Constructor (__init__):** Initialize object attributes during object creation.
- **Static Methods:** Implement utility operations that do not depend on object state.
- **Class Methods:** Perform class-level operations and object creation.
- **Context Managers (with Statement)** – Automatically manage file resources while reading datasets, writing reports, and saving or loading trained models.
- **Exception Handling:** Handle runtime errors and invalid operations gracefully.
- **File Handling:** Read datasets and generate reports, model files, and prediction history.
- **Regular Expressions (Regex):** Validate user inputs before processing.

13. MVC Architecture

The application follows the **Model-View-Controller (MVC)** architectural pattern to achieve separation of concerns and improve maintainability.

- **Model** – Represents business entities and manages application data.

- **View** – Displays menus, accepts user input, and presents prediction results.
- **Controller** – Receives user requests and coordinates communication between the View and Services.
- **Services** – Implement business logic, data preprocessing, model training, prediction, and report generation.
- **Validation** – Validate user inputs and dataset integrity before processing.
- **Utilities** – Provide reusable helper functions, constants, logging, and file management.
- **Loose Coupling** – Modules operate independently, reducing dependencies.
- **High Cohesion** – Each module performs a single, well-defined responsibility.

14. Input and Output Specification

The system accepts historical flight data for model training and flight details from users for fare prediction. Based on the processed dataset and trained Machine Learning model, the application generates predictions and maintains supporting records for future analysis.

Input

- Historical Flight Dataset (.csv)
- Airline
- Source City
- Destination City
- Journey Date
- Departure Time
- Arrival Time
- Flight Duration
- Total Stops
- Additional Information

Output

- Predicted Flight Fare
- Trained Machine Learning Model (.pkl)
- Prediction History (.csv)

- Model Evaluation Report
- Performance Metrics (MAE, RMSE, R^2 Score)
- Generated Reports (.pdf/.csv)

15. Conclusion

The Flight Fare Prediction System demonstrates the practical application of Machine Learning for solving a real-world regression problem. The project integrates data preprocessing, feature engineering, model training, evaluation, and prediction within a modular software architecture.

By combining Machine Learning techniques with Object-Oriented Programming and MVC architecture, the system provides accurate fare estimation while maintaining scalability, maintainability, and code reusability.

The expected outcome of the project is a reliable application capable of predicting flight fares based on historical data, enabling users to make informed travel decisions. The project also provides a strong foundation for future enhancements such as real-time airline data integration, web deployment, advanced prediction models, and additional analytical features.

15. References

1. **Dataset** - Data_Train.csv and Test_set.csv
<https://www.kaggle.com/datasets/harshavarshney/flight-fare>